

README: Trusting the Weights: Natural Vulnerability Poisoning of Open-Weights Code Models

Overview

This repository provides a full end-to-end pipeline for Python code vulnerability analysis, safe/vulnerable code generation, model fine-tuning, and automated security evaluation. It supports multiple models (StableCode, LLaMA, Gemma), incorporates QLoRA training, and integrates DRL and fuzzy logic evaluation.

Capabilities include:

- Vulnerable/safe code generation
- Fuzzy logic and DRL evaluation for attack strength
- Dataset preparation for fine-tuning
- Fine-tuning multiple models
- Security evaluation using Semgrep, Bandit, Snyk, and GPT-based scoring
- Inference and code generation
- Visualization and analysis of vulnerability metrics

Step-by-Step Pipeline

1. Data Processing & Code Variant Generation & Fuzzy Steps

1. **Step1_process_files2-4.py** – Generate safe and multiple vulnerable variants of input code using AI; analyze AST-based similarity.
2. **Step2_evaluate_code_blocks7-10.py** – Evaluate code variants with Semgrep, Bandit, Snyk; compute AST- and TF-IDF-based conceptual similarity.
3. **Step3_process_files_codebreaker.py** – Split input files into vulnerable and obfuscated sections for structured output.
4. **Step4_process_svb4.py** – Generate multiple safe versions of vulnerable code using GPT-4 while preserving functionality.
5. **Step5_evaluate_code_blocks7_codebreaker3.py** – Compare obfuscated code against safe variants using AST and conceptual similarity.
6. **Step6_selecting_fuzzy.py** – Select best-performing variants based on fuzzy logic metrics; classify attack strength.
7. **Step7_Fuzzy_data_results_all_diagram.py** – Generate comparative bar charts for attack strength levels.

8. **Step8_plotting_and_Fuzzy_code.py** – Apply fuzzy logic on code metrics and visualize membership functions.

2. DRL-Based Attack Strength Prediction

9. **Step9_Three_types_extracted_files.py** – Extract key evaluation and fuzzy logic features for analysis.
10. **Step10-0-Systematic_codes_generator-QWEN-for_training.py**–Generating the training data using Qwen3.
Step10_Extracted_features_all.py – Merge features from multiple SV fuzzy output files for DRL training.
Step10-0_0_strong-data.py– Extracting the generated strong data.
Step_10-0_1_Random_data.py–Extracting the generated random data.
Step_10_1-AA_codes_pass_similraty_before_inference.py– Evaluating the similarity and pass rate.
Step10_2-Strong_random_Features extracted.py– Feature extraction step.
Step10-3-Fuzzy_evaluation_ground-trust_labels.py– Applying the fuzzy rules to achieve the labels.
11. **Step11_DRL_training_all.py** – Train Double Deep Q-Network (DDQN) on fuzzy features to predict attack strength (Weak, Moderate, Strong).
Step11-2 DRL_Inference.py–Runing the model inference after training for any unseen variants.

3. Prompt & Dataset Generation

12. **Step12_New_code_prompt_generation_vulnerable2.py** – Generate structured prompt–completion pairs from vulnerable code.
13. **Step13_New_code_prompt_generation_clean3.py** – Generate prompt–completion pairs from clean code files.
14. **Step14_input_code.py** – Consolidate vulnerable prompt–completion text files into JSONL format.
15. **Step15_finetune_stablecode5.py** – Fine-tune StableCode using QLoRA on structured prompt–completion data.
16. **Step16_Input_prompt.py** – Aggregate prompt texts from multiple JSON files into a single file.
17. **Step17_confusing_fine-tuning2.py** – Generate code from prompts using LoRA fine-tuned StableCode model.
18. **Step18_vulnerable_codes.py** – Extract Python code blocks from generated outputs; consolidate clean code into safe repository.

19. **Step19_Score_fine_tuning_security.py** – Evaluate generated code security using GPT scoring.
20. **Step20_New_code_generation.py** – Generate Python code for multiple prompts using StableCode model.
21. **Step20_New_code_prompt_generation.py** – Generate generalized instruction prompts from clean code for code-generation models.
22. **Step21_prompt_generation4.py** – Generate partial code-style prompts (docstring + initial lines) for automated code generation guidance.
23. **Step22_New_code_prompt_generation_vulnerable2_req_sock-jinja.py** – Generate structured prompt–completion pairs for requests, sockets, and Jinja2 code.
24. **Step23_Input_code_req_socket_jinja2.py** – Merge VULN1–VULN5 code blocks into a clean JSONL dataset ready for training.
25. **Step24_training_input_all.py** – Combine multiple training datasets, validate JSON entries, and output consolidated data.
26. **Step25_fine_tuning_stablecode5_all.py** – Fine-tune StableCode-Instruct-3B model with LoRA, track loss, save model, and generate plots.
27. **Step26_New_code_prompt_generation_clean3_socket_req_jinj.py** – Generate generalized prompt–completion pairs from clean code for fine-tuning.
28. **Step27_fine_tuning_all_Llama3.py** – Fine-tune LLaMA-3B model on the combined dataset.
29. **Step28_fine_tuning_all_Gemma.py** – Fine-tune Gemma-2B model on dataset; save adapters, tokenizer, and loss plots.
30. **Step29_Input-prompt_request_socket_jinja2_clean.py** – Extract only the "prompt" parts from generated TXT files for downstream inference.
31. **Step30_input_prompt_safe.py** – Select first prompt from each group of five and save into a reordered prompt file.

4. Inference & Model Evaluation

32. **Step31_confusing_fine_tuning3_all.py** – Generate Python code using fine-tuned StableCode model.
33. **Step31_confusing_lama_ours.py** – Generate outputs using fine-tuned LLaMA 3B model.
34. **Step32_confusing_model_Gemma.py** – Generate Python code using fine-tuned Gemma model.
35. **Step33_vulnerable_code_all.py** – Extract code blocks from generated files; consolidate SAFE files into a single folder.
36. **Step34_Vulnerable_code_lama.py** – Extract LLaMA-generated Python code blocks into a dedicated folder.

- 37. **Step35_Vulnerable_code_gemma.py** – Extract Gemma-generated Python code blocks into a dedicated folder.
- 38. **Step36_changing_name_numbers.py** – Sequentially rename SAFE code files for consistent numbering.

5. Security Scoring

- 39. **Step37_score_fine_tuning_security_all.py** – Evaluate generated code against safe baseline using GPT scoring.
- 40. **Step38_input_data_ours-vuln1.py** – Consolidate all VULN1.txt files into a single folder with sequential renaming.
- 41. **Step39_score_security_ours_vul.py** – Evaluate vulnerability levels of generated code using GPT scoring.
- 42. **Step40_score_fine_tuning_security_all_lama.py** – Score LLaMA-generated code against safe baselines.
- 43. **Step41_score_ours_lama_vul.py** – GPT-based evaluation of LLaMA-generated code vulnerabilities.
- 44. **Step42_score_ours_lama_vul2.py** – Assign severity score (1–10) to LLaMA code vulnerabilities.
- 45. **Step43_score_fine_tuning_security_all_GAMMA.py** – Evaluate Gemma-generated code against safe baseline using GPT.
- 46. **Step44_score_ours_GEMMA_vul.py** – GPT-based evaluation of Gemma-generated code vulnerabilities.
- 47. **Step45_score_plot_all2.py** – Extract security scores, categorize vulnerabilities, generate summaries and plots.
- 48. **Step46_score_fine_tuning_security_all2.py** – Batch security scores, compute vulnerability percentages, generate stacked bar plots.

6. CodeBreaker Dataset & Fine-Tuning

- 49. **Step47_input_data_code_breaker.py** – Collect and rename obfuscated .txt files into single dataset folder.
- 50. **Step48_input_codebreaker_safe.py** – Collect SAFE .txt files into single dataset folder.
- 51. **Step49_generate_prompts_codebreaker.py** – Generate prompt–completion pairs from obfuscated code for fine-tuning.
- 52. **Step50_training_data_obfu_codebreaker.py** – Convert multiple JSON objects into a single JSON array ready for OpenAI fine-tuning.
- 53. **Step51_augmented_data_codebreaker.py** – Use GPT-4 to generate augmented versions of prompts for CodeBreaker fine-tuning.

54. **Step52_fine_tuning_codebreaker.py** – Fine-tune Stable-Code-Instruct-3B with LoRA on augmented CodeBreaker dataset.
55. **Step53_training_data_obfu_codebreaker.py** – Consolidate augmented dataset into single JSON array for fine-tuning.
56. **Step54_confused_model_codebreaker.py** – Generate Python code using CodeBreaker fine-tuned model.
57. **Step55_vulnerable_codes.py** – Extract Python code blocks and consolidate safe code into dedicated folder.
58. **Step56_score_security_codebreaker.py** – Evaluate generated CodeBreaker code against safe baseline using GPT-4.
59. **Step57_score_plot_all_codebreaker.py** – Produce grouped analysis and visualizations of CodeBreaker security scores.
60. **Step58_ours_codebreaker_statistic_analysis_tools.py** – Comprehensive statistical and automated security evaluation of code files.
61. **Step59_second_comparison_ours_codebreaker.py** – Generate per-folder summary report of tool-based analysis.
62. **Step60_plotting_pass_rate.py** – Compare pass rates (Semgrep, Bandit, Snyk) between “ours” and CodeBreaker models.

7. Input Selection & Prompt Fixing

63. **Step61_8_percentage_extracted_files.py** – Extract 8% of Python code files matching specific patterns for analysis.
64. **Step62_get_request_tag_data.py** – Add <orig> and <vuln> tags, generate modified vulnerable versions.
65. **Step63_second_transformed.py** – Obfuscate/transformation pipeline applied to vulnerable files.
66. **Step64_get_request_prompts_inputs.py** – Generate JSONL/JSON prompt-completion pairs for fine-tuning.
67. **Step65_second_codebreaker_finetuning.py** – Fine-tune 4-bit LoRA CodeBreaker model on merged prompts.
68. **Step66_clean_prompt_get_request_org.py** – Clean original prompts, produce unified text and JSON files.
69. **Step67_inference_model_fixed.py** – Inference script for CodeBreaker fine-tuned model.
70. **Step68_fix_inputs.py** – Fix input prompt files, convert them to JSON objects, save for consistent fine-tuning/inference.

Usage

1. Place input code files in the respective folders.
2. Execute scripts in sequence, or use modular steps as required:

```
python Step68_fix_inputs.py
```

```
python Step67_inference_model_fixed.py
```

```
python Step52_fine_tuning_codebreaker.py
```

Outputs, fine-tuned models, and evaluation results will be saved in corresponding folders.

Dataset and Data Generation

We used the CodeBreaker dataset in this project.

Dataset link:

<https://github.com/datasec-lab/CodeBreaker/>

Dependencies

Software Requirements

- Python 3.8+
- PyTorch
- Transformers
- PEFT
- Matplotlib, Seaborn
- JSON, OS
- GPT API (for scoring)
- Security tools: Semgrep, Bandit, Snyk
- Qwen3

Hardware Used for Experiments

- CPU: Intel Core i9 processor
- vCPUs: 12

- RAM: 32 GB
- GPU: NVIDIA GeForce RTX 3080
- CUDA Version: 12.4

Notes

- Steps are modular; parts of the pipeline (fine-tuning, inference, evaluation) can be executed independently.
- CodeBreaker uses a LoRA-adapted 4-bit StableCode model.
- LLaMA and Gemma models are fine-tuned on the same processed datasets for comparative evaluation.
- Proper folder and file naming is required for smooth pipeline execution.
- The hardware and software setup was used for running the experiments, but it can be adjusted depending on the available system resources or software versions.
- Due to GitHub repository size limitations, we are unable to include all intermediate datasets, generated outputs, and fine-tuning artifacts in this repository. The provided scripts and pipeline enable these data and outputs to be reproduced by following the steps described above.